A Hybrid Retrieval-Augmented Generation Framework for Enterprise Knowledge Management

Project Name: KNOWX

A Mini Project Report submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

By

B. Manaswini – 2420080029 (Team Lead)

G. Babitha – 2420080069

G. Rohan – 2420030564

P. Adithya Ram – 2420030383

Team Name: KNOWX

Under the Esteemed Guidance of

Dr. K. Swanthana



K L (Deemed to be) University

Academic Year 2026–2027

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

DECLARATION

The mini-project report entitled "**A Hybrid Retrieval-Augmented Generation Framework for Enterprise Knowledge Management**" is a record of bona fide work carried out by B. Manaswini (2420080029), G. Babitha (2420080069), G. Rohan (2420030564) and P. Adithya Ram (2420030383) in partial fulfilment of the requirements for the award of the B.Tech degree at K L (Deemed to be) University. The work presented in this report is original to the best of our knowledge, and all datasets, tools, code and external sources used have been appropriately acknowledged.

B. Manaswini – 2420080029
G. Babitha – 2420080069
G. Rohan – 2420030564
P. Adithya Ram – 2420030383

K L (Deemed to be) University

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



CERTIFICATE

This is to certify that the mini-project report entitled "**A Hybrid Retrieval-Augmented Generation Framework for Enterprise Knowledge Management**" is a bonafide work carried out by B. Manaswini (2420080029), G. Babitha (2420080069), G. Rohan (2420030564) and P. Adithya Ram (2420030383) under the supervision of Dr. K. Swanthana during the academic year 2026–2027, in partial fulfilment of the requirements for the award of the Bachelor of Technology degree at K L (Deemed to be) University.

Signature of the Project Supervisor

Signature of the HODSignature of the External Examiner

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who assisted us in one way or another in the completion of our project.

Our greatest appreciation goes to our guide, Dr. K. Swanthana, Department of Artificial Intelligence and Data Science, whose tremendous support, encouragement and guidance for this project cannot be expressed in words.

We express our gratitude to Dr. Sreenath, Head of the Department of Artificial Intelligence and Data Science, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff, and all who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

B. Manaswini – 2420080029
G. Babitha – 2420080069
G. Rohan – 2420030564
P. Adithya Ram – 2420030383

TABLE OF CONTENTS

S.No.	Contents	Page No.
1.	Abstract	6
2.	Introduction	8
3.	Literature Survey	10
4.	Problem Statement and Objectives	14
5.	Hardware and Software Requirements	15
6.	Implementation	16
7.	Results and Discussion	22
8.	Conclusion and Future Scope	27
9.	References	29

1. ABSTRACT

Enterprise documents such as annual reports and technical filings are long, and finding a specific answer in them by hand is slow. This project presents KNOWX, an enterprise document question-answering system built on hybrid retrieval and Retrieval-Augmented Generation (RAG). The dataset consists of nine enterprise documents collected from Microsoft, NVIDIA and Infosys, containing approximately 5,000,829 extracted characters. Their text was extracted and divided into 6,256 chunks. Each chunk is represented in two ways: as a 384-dimensional sentence embedding created with all-MiniLM-L6-v2 and stored in a FAISS index, and as an entry in a BM25 keyword index. A user question is searched against both indexes, the two ranked lists are merged with Reciprocal Rank Fusion (RRF), and a cross-encoder reranks the merged passages. The best passages are assembled into a context with source information and passed to Llama 3 8B Instruct, a quantised (Q4_K_M) GGUF model that runs locally on the CPU through llama-cpp-python. The generated answer is returned along with supporting source information. The backend retains document names, chunk identifiers and retrieval scores, while the user interface displays the source document name and organization for easier reading. No model was trained, because all models are pre-trained. The integrated pipeline ran successfully from the command line, and a test question about Microsoft's business and financial performance produced an answer with five source entries.

Team Name: KNOWX

Team Members and Responsibilities:

Team Member	Role	Technical Responsibilities
B. Manaswini 2420080029 (Team Lead)	Project Lead & System Integration	Overall project planning and coordination; design of the end-to-end architecture; setting up the quantised Llama 3 8B Instruct (GGUF, Q4_K_M) model and running it locally through llama-cpp-python; integrating ingestion, retrieval, reranking and generation into one pipeline (src/pipeline.py); end-to-end testing of the integrated system.
G. Babitha 2420080069	Backend & API Development	Developing the FastAPI backend (app/app.py) served with Uvicorn; designing the API that receives questions and returns answers with source entries; loading the pipeline in the backend; context building and formatting of source information in API responses; testing of the backend and API.
G. Rohan 2420030564	Frontend Development	Developing the browser front end (app/static/app.js); building the question input and the display of generated answers and source entries; connecting the front end to the backend API; testing the interface in the browser.
P. Adithya Ram 2420030383	Research, Data Processing & Retrieval Pipeline	Research on retrieval techniques; collecting the enterprise documents; document ingestion, text extraction and chunking; creating the embeddings and the FAISS index; building the BM25 index; implementing semantic and BM25 retrieval (src/hybrid_search.py), Reciprocal Rank Fusion and cross-encoder reranking (src/reranker.py); testing of the retrieval modules.

Dataset/source and project repository:

Dataset: enterprise documents from Microsoft, NVIDIA and Infosys, collected by the project team. The dataset is described in Section 6.1 and Section 7.

Project repository:

The source code and project implementation are available on GitHub:
https://github.com/Babitha17/KLH-AIDS-2026-27-2420080069-Knowx

2. INTRODUCTION

Large organisations keep much of their knowledge in long documents such as annual reports, financial filings and technical papers. These documents are written to be read carefully, not

searched quickly. A person who wants to know how a company describes its business or reports its financial results has to open several files, look for keywords and read many pages before finding a reliable answer. When the collection grows to several documents from several companies, this manual work becomes slow, inconsistent and difficult to repeat.

Machine learning and natural language processing can reduce this effort because they allow a computer to work with the meaning of text and not only with exact words. Keyword ranking functions such as BM25 are fast and precise when a question uses the same terms as the document, but they fail when the wording is different. Sentence embeddings place passages with similar meaning close together in a vector space, which helps with paraphrased questions, but they can miss exact names, figures and rare terms. Retrieval-Augmented Generation (RAG) adds a language model to retrieval: the model receives the retrieved passages and writes an answer from them, so the answer is tied to the documents and can point back to its sources.

This project, **KNOWX**, applies these ideas to enterprise knowledge management. It is a document question-answering system that combines semantic retrieval, BM25 keyword retrieval, Reciprocal Rank Fusion (RRF) , cross-encoder reranking and a locally running Llama 3 8B Instruct model . KNOWX is not a conventional text classification project. Parts of the usual report structure that assume labelled data, such as class distributions, train/test splits and confusion matrices, are therefore adapted to this project, and each adaptation is explained where it occurs.

Target users and intended use

The intended users are analysts, managers, employees and students who need quick, source-backed answers from a fixed collection of company documents. KNOWX is meant to help a user find and read the relevant passages faster. Its answers should still be checked against the cited sources before they are used for any decision.

Input and output

The input is a question written in natural language, for example, “What does Microsoft report about its business and financial performance?” The output is not a class label. The output consists

of a generated text answer accompanied by source information. The backend maintains the document name, chunk identifier and retrieval score for each retrieved passage. The browser interface displays the source document name and organization, keeping the source section simple and readable.

Proposed pipeline

Enterprise documents from Microsoft, NVIDIA and Infosys are collected, and their text is extracted and split into chunks. Each chunk is turned into a 384-dimensional embedding and stored in a FAISS index, and a BM25 index is built for keyword search. At question time, both indexes are searched, the results are fused with RRF, and a cross-encoder reranks them. The selected passages form a context for Llama 3 8B Instruct, which generates the answer. A FastAPI application and a browser front end display the answer and its sources. No model training was needed because every model used is pre-trained, and evaluation so far consists of an end-to-end test run.

Organisation of the report

Section 3 reviews related work. Section 4 states the problem and the objectives. Section 5 lists the hardware and software used. Section 6 describes the implementation, including the workflow diagram and the modules. Section 7 reports the results that were actually obtained and explains which standard text analytics results could not be produced. Section 8 gives the conclusion and future scope, and Section 9 lists the references.

3. LITERATURE SURVEY

Ten sources were reviewed. They cover the main parts of a hybrid RAG system: retrieval-augmented generation, keyword ranking, sentence embeddings, rank fusion, vector search, reranking and the local language-model runtime. Each summary gives the problem, the method,

the evaluation, the main finding and a limitation. Numbers in brackets refer to the list in Section 9.

[2] Lewis et al. (2020), Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Language models store knowledge in their parameters, which makes it hard to update that knowledge or to show where an answer came from. Lewis et al. pair a pre-trained sequence-to-sequence generator with a dense vector index of Wikipedia that is searched by a pre-trained neural retriever. They compare using the same passages for a whole answer with using different passages for each token, and they test on open-domain question answering and other knowledge-intensive tasks. The models set the state of the art on three open-domain QA tasks and produced more specific and factual text than a parametric-only baseline. The index covers Wikipedia only, so private enterprise documents are not studied.

[3] Robertson and Zaragoza (2009), The Probabilistic Relevance Framework: BM25 and Beyond

Robertson and Zaragoza address how documents should be ranked for a query when relevance is treated as a probability. The monograph describes the Probabilistic Relevance Framework and the ranking functions that follow from it, including BM25 and BM25F, and discusses their relation to other statistical retrieval models and the tuning of free parameters. As a theoretical work, it does not rely on one dataset. It presents BM25 as one of the most successful text-retrieval algorithms. BM25 scores matching terms, so it cannot find a passage that answers a question in different words, and the framework assumes that relevance is binary.

[4] Reimers and Gurevych (2019), Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks

BERT compares two sentences by reading them together, which makes searching a large set of sentences very slow. Reimers and Gurevych modify a pre-trained BERT network with siamese and triplet structures so that each sentence gets its own embedding and embeddings can be compared with cosine similarity. They evaluate on semantic textual similarity and transfer tasks. Finding the most similar pair fell from about 65 hours with BERT to about 5 seconds while accuracy was maintained, and the method outperformed other sentence-embedding approaches on these tasks. The embeddings represent short texts, so long documents must be split into chunks before they are encoded.

[5] Cormack, Clarke and Büttcher (2009), Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods

Combining the rankings of several retrieval systems normally needs training data or careful score tuning. Cormack, Clarke and Büttcher propose Reciprocal Rank Fusion (RRF), which gives each document a score based on its rank position in every list and then sorts by the total. They test it on TREC collections and on the LETOR 3 learning-to-rank data. RRF did better than Condorcet Fuse and, in most of their comparisons, better than the best single system. The paper is only two pages long and uses older benchmark collections. RRF uses rank positions only, so differences in the original scores are ignored.

[6], [10] Johnson, Douze and Jégou (2021), Billion-scale similarity search with GPUs

Searching very large collections of vectors is costly, and earlier GPU methods made poor use of the hardware. Johnson, Douze and Jégou design a GPU algorithm for k-selection and apply it to brute-force, approximate and compressed-domain search based on product quantisation. The work is released as the FAISS library [10]. They test on large collections such as Yfcc100M image vectors. The implementation was up to 8.5 times faster than the previous GPU state of the art and built a k-nearest-neighbour graph of 95 million images in 35 minutes. These gains target very large collections and GPUs, whereas KNOWX searches 6,256 vectors on a CPU.

[7] Nogueira and Cho (2019), Passage Re-ranking with BERT

A first-stage retriever returns passages that are not all relevant, so a second stage has to judge them more carefully. Nogueira and Cho use BERT as a passage re-ranker: the query is given as sentence A and the passage as sentence B, and the model estimates how likely the passage is to be relevant. They evaluate on MS MARCO and TREC-CAR. The system was the state of the art on TREC-CAR and the top entry on the MS MARCO leaderboard, improving MRR@10 by 27% relative to the previous best. Each candidate is scored separately and at higher cost than in the first stage, so the method is used on a short candidate list, and it was tested on only two datasets.

[8] Sentence Transformers documentation, Retrieve & Re-Rank

The documentation describes a two-stage design for search. A fast retriever, such as a bi-encoder or BM25, collects candidates, and a cross-encoder then re-ranks them. It explains that cross-encoders attend to the query and the document together, which gives better quality but is too slow to run over a whole collection, so they are applied to a limited set of candidates, for example the top 100. The examples use Simple English Wikipedia and NanoNFCorpus, and one script

combines dense, sparse and fused results with RRF. This is practical documentation, not a peer-reviewed study, and its example data differ from enterprise reports.

[9] Sentence Transformers, all-MiniLM-L6-v2 model card

The model card describes all-MiniLM-L6-v2, a sentence-transformers model that maps sentences and paragraphs to a 384-dimensional vector space for semantic search and clustering. It starts from the pre-trained MiniLM-L6-H384-uncased model and is fine-tuned with a contrastive objective on more than one billion sentence pairs. The card refers readers to the Sentence Embeddings Benchmark instead of giving its own figures. By default, input longer than 256 word pieces is truncated, so chunks need to be short, and the training data are general-purpose rather than specific to enterprise reports.

[11] Meta AI (2024), Introducing Meta Llama 3

Meta announced Llama 3 on 18 April 2024 as the next generation of its openly available language models, released in 8B and 70B sizes with instruction-tuned versions. The announcement reports benchmark results and a human evaluation set of 1,800 prompts across 12 use cases, and introduces safety tools such as Llama Guard 2, Code Shield and CyberSec Eval 2. Because the model weights are available, they can be run locally, which suits a document system that should stay on one machine. The results are published by Meta itself, and the announcement does not test question answering over enterprise documents or the quantised form used here.

[12] llama-cpp-python documentation

The documentation describes Python bindings for the llama.cpp library. They provide a high-level Llama class, a low-level interface to the C API and an OpenAI-compatible server. The API reference lists the settings used in this project: n_ctx for the context window, n_threads for CPU threads and n_gpu_layers for the number of layers offloaded to a GPU, which defaults to 0. The documentation shows how to load a quantised GGUF model on ordinary hardware, but it is not a benchmark, so it says nothing about answer quality or speed on a particular laptop.

Table 1: Comparison of the reviewed approaches

Approach	What it does	Strength	Limitation for this use
BM25	Ranks chunks by matching terms	Precise for exact names and terms	Misses differently worded questions

Approach	What it does	Strength	Limitation for this use
Sentence embeddings	Maps text to vectors compared by cosine similarity	Matches meaning, not only words	Short inputs; may miss exact terms
RRF	Merges several ranked lists using rank positions	Needs no training data	Ignores score magnitudes
FAISS	Fast nearest-neighbour search over vectors	Scales to very large collections	Main gains are at large scale and on GPUs
Cross-encoder	Reads question and passage together to rerank	More accurate than bi-encoders	Too slow for a whole collection
RAG	Generates an answer from retrieved passages	Grounded answers; sources can be shown	Studied on Wikipedia, not enterprise documents
Llama 3 with llama-cpp-python	Runs an open LLM locally on CPU	Runs on a laptop; no external service	Quality and speed not measured here

Gap addressed by this mini-project

The reviewed sources treat retrieval, fusion, reranking, embeddings, vector search and local language-model inference mostly as separate topics. They are usually tested on benchmark collections such as Wikipedia or MS MARCO, and several assume a GPU. None of the sources reviewed describes a complete pipeline of this kind running on a CPU-only laptop over company documents and showing its sources to the user. KNOWX addresses this practical integration gap. It does not claim to perform better than the systems above, because no comparison has been carried out.

4. PROBLEM STATEMENT AND OBJECTIVES

Problem Statement

To build a question-answering system that finds the relevant passages in a collection of enterprise documents through hybrid retrieval and returns a source-supported answer generated by a locally running large language model.

Problem Description

Enterprise reports are long, and employees or analysts who need one fact from them must search and read manually. Keyword search alone misses questions that are worded differently from the text, and a language model used without retrieval can state facts that are not in the documents and cannot show a source. KNOWX retrieves the relevant chunks first, generates the answer from them, and shows the document name, chunk identifier and score for each source. Running the language model locally keeps the whole pipeline on one machine.

Objectives

- Collect and inspect a set of enterprise documents and record its source, size and structure: nine documents from Microsoft, NVIDIA and Infosys, divided into 6,256 chunks. A class distribution is not recorded because the data have no labels.
- Extract text from the documents and split it into smaller chunks that keep the source document and chunk identifier.
- Represent each chunk with a 384-dimensional all-MiniLM-L6-v2 embedding stored in FAISS and with a BM25 keyword index.
- Implement hybrid retrieval that searches both indexes, combines the results with Reciprocal Rank Fusion and reranks them with a cross-encoder.
- Build a context from the best passages and generate an answer with Llama 3 8B Instruct (GGUF, Q4_K_M) running on a CPU through llama-cpp-python.
- Run the integrated pipeline end to end and record its output, including the sources returned. A formal quantitative evaluation has not yet been performed and is listed as future work.
- Provide a FastAPI application and a browser front end that send questions to the backend and display answers with their sources.
- Discuss limitations, bias, privacy and possible improvements (Sections 7 and 8).

The generic objectives of training and comparing classifiers and of reporting accuracy, precision, recall, F1-score and a confusion matrix are not included, because this project has no labelled dataset and trains no classifier. Section 7 explains this further.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements

The project was developed and tested on a single laptop. No NVIDIA CUDA GPU was used, and the language model runs entirely on the CPU. Internet access was needed to download Python packages and the model files.

Table 2: Development hardware

Component	Specification
Operating system	Windows 11
Processor	AMD Ryzen 5 8640HS
Memory (RAM)	16 GB
Storage	SSD
Graphics	Integrated AMD Radeon Graphics
GPU acceleration	None. No NVIDIA CUDA GPU was used; GPU layers for the language model were set to 0
CPU threads used by the language model	6

Software Requirements

The project uses two Python environments, named venv and hf-venv. The hf-venv environment was used for the local Llama 3 setup, and its recorded Python version is 3.13.2. Hugging Face was used only to authenticate and to download the model files; the quantised model is loaded and executed locally with llama-cpp-python, and the Hugging Face Transformers inference pipeline is not used. Version details of the individual libraries were not recorded during implementation. The main software components are listed in Table 3.

Table 3: Software components

Component	Details	Version / status
Python	Environments venv and hf-venv (local Llama 3 setup in hf-venv)	3.13.2 (hf-venv). The version used in venv was not recorded
Language model	Meta-Llama-3-8B-Instruct-Q4_K_M.gguf (GGUF format, Q4_K_M quantisation)	Context window 4096 tokens

Component	Details	Version / status
Model runtime	llama-cpp-python (not the Hugging Face Transformers library)	Version details were not recorded during implementation
Embedding model	all-MiniLM-L6-v2, 384-dimensional embeddings, used through the Sentence Transformers library	Version details were not recorded during implementation
Vector index	FAISS (models/semantic_index.faiss)	Version details were not recorded during implementation
Keyword index	BM25 using the rank-bm25 library (models/bm25_index.pkl)	Version details were not recorded during implementation
Reranker	Cross-encoder cross-encoder/ms-marco-MiniLM-L-6-v2 [13] (src/reranker.py)	Pre-trained checkpoint
Backend	FastAPI application (app/app.py) served with Uvicorn	Version details were not recorded during implementation
Other libraries	NumPy and Requests	Version details were not recorded during implementation
Front end	JavaScript (app/static/app.js)	Runs in a web browser

6. IMPLEMENTATION

6.1 Data and Task

The task is question answering over a fixed set of documents. Given a natural language question, the system retrieves the most relevant passages and generates an answer from them. This is not label prediction, so the data have no target labels.

The dataset is a collection of nine enterprise documents obtained from Microsoft, NVIDIA and Infosys. The project record names, for example, MSFT_FY24Q4_10K.docx and Microsoft annual reports.docx. After ingestion, the collection contained 6,256 text chunks: 893 from Microsoft, 2,453 from NVIDIA and 2,910 from Infosys. There is no train/validation/test split, because no model was trained or tuned. All 6,256 chunks form the searchable knowledge base. Usage of the documents is limited to the purposes of this academic project.

6.2 NLP Preprocessing and Features

Text was extracted from each document and divided into smaller chunks. The source document name and a chunk identifier are kept with every chunk, so that each retrieved passage can be traced back to its origin. The nine documents contained approximately 5,000,829 extracted characters,

which were divided into 6,256 chunks. This gives roughly 800 characters of extracted text per chunk on average (calculated from these two figures). The chunk boundaries were set in the document-processing step, and the individual chunk-size and overlap settings are not stated in this report.

Two text representations are created from the same chunks. For semantic retrieval, all-MiniLM-L6-v2 converts each chunk into a 384-dimensional embedding. The vectors are stored in a FAISS index (models/semantic_index.faiss) with a metadata file (models/semantic_metadata.json). For keyword retrieval, a BM25 index is built over the chunks with the rank-bm25 library and saved as models/bm25_index.pkl. BM25 works on the tokenised words of each chunk, and the exact cleaning steps applied before indexing are not stated in this report. The two representations support each other: embeddings help when a question is worded differently from the text, and BM25 helps when it contains exact names and terms.

6.3 Model Training and Evaluation

No supervised training was carried out. The embedding model, the cross-encoder and Llama 3 8B Instruct are used as released. For this reason, there was no hyperparameter search, validation strategy or best-model selection, and the risk of data leakage from fitting preprocessing on test data does not arise. The indexes are built on the whole collection because they are the knowledge base and not a learned model. A random seed is not applicable to retrieval. The generation settings other than the context window, thread count and GPU layers (Table 5) are not stated in this report. The system has so far been evaluated by running the integrated pipeline end to end; the result is given in Section 7.

6.4 Project Workflow and Modules

Figure 1 shows the workflow of KNOWX. Steps 1 to 4 build the knowledge base, and steps 511 answer a question.

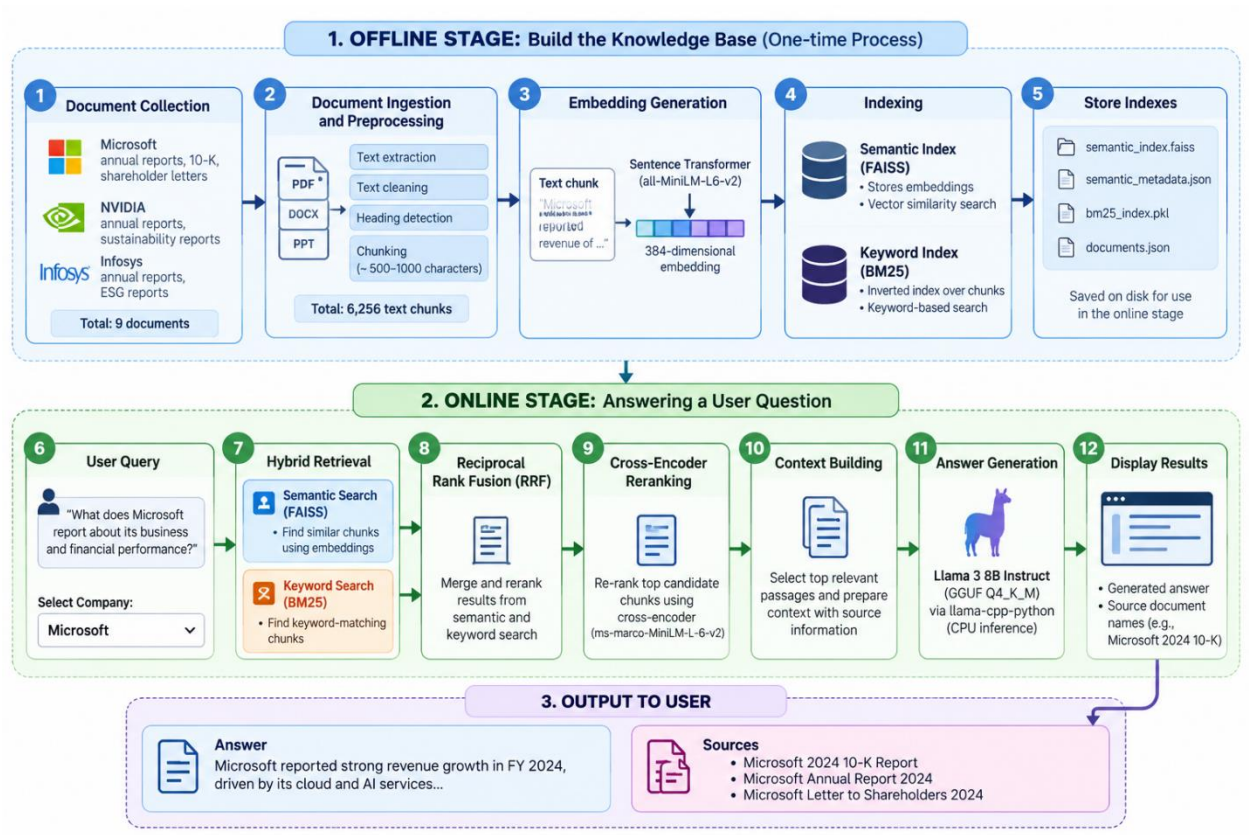


Figure 1: Workflow of the KNOWX pipeline (steps 1 to 12)

The project is organised into the following folders and files under C:\Users\manas\KnowX\KNOWX.

Table 4: Project files and their purpose

File / folder	Purpose
venv/ and hf-venv/	Python virtual environments; hf-venv was used for the local Llama 3 setup

File / folder	Purpose
data/raw/	Original collected enterprise documents (Word files such as MSFT_FY24Q4_10K.docx and Microsoft annual reports.docx)
data/processed/ and data/documents.json	Processed document data produced by ingestion, from which the chunks are created
models/semantic_index.faiss	FAISS index of the 384-dimensional chunk embeddings
models/semantic_metadata.json	Metadata that accompanies the semantic index
models/bm25_index.pkl	BM25 keyword index built with rank-bm25
models/llama3-8b-q4/ (Meta-Llama-3-8B-Instruct-Q4_K_M.gguf)	Quantised Llama 3 8B Instruct model file loaded locally by llama-cpp-python
src/pipeline.py	Coordinates retrieval, context building and answer generation; can be run from the command line
src/hybrid_search.py	Hybrid retrieval (semantic and BM25 search)
src/reranker.py	Cross-encoder reranking of the fused candidate passages
app/app.py	FastAPI application that receives questions and returns answers with sources
app/static/app.js	Front end: sends questions to the backend and shows answers and sources

The ingestion and index-building steps (text extraction, chunking, embedding, and creation of the FAISS and BM25 indexes) produce the contents of data/processed/, data/documents.json and the models/ folder listed above.

6.5 Machine-Learning Implementation

Because no model was trained, there is no baseline or comparison model. The implementation consists of the following pre-trained components working together.

Semantic retrieval. The question is converted into an embedding with the same all-MiniLM-L6-v2 Sentence Transformers model, and the closest chunk vectors are found in the FAISS index .

Keyword retrieval. The question is also scored against the BM25 index, which ranks chunks by how well their terms match the question .

Reciprocal Rank Fusion. The two ranked lists are combined with RRF . A chunk receives a score equal to the sum, over the two lists, of 1 / (k + rank), where rank is its position in that list and k is a constant. Chunks that rank high in both lists rise to the top. The numerical settings (the value of

k, the retrieval depth and the number of passages sent to the language model) are not stated in this report; Table 6 summarises the role of each parameter.

Cross-encoder reranking. The reranker is the pre-trained model cross-encoder/ms-marco-MiniLM-L-6-v2, whose model card states that it was trained on the MS MARCO passage ranking task. Unlike the two first-stage retrievers, which score chunks independently, it reads the question and a candidate passage together and gives a relevance score, which is more accurate than comparing separate embeddings but too slow to run on every chunk , . It is therefore applied to the fused candidates, and the highest-scoring passages are passed on to build the context.

Context building. The relevant passages are selected and assembled into a context that includes the source information of each passage. The 4096-token context window limits how much retrieved text can be given to the model together with the question and the generated answer.

Answer generation. The selected passages are given to Llama 3 8B Instruct as context for answering the question, so that the answer is grounded in the enterprise document collection. Hugging Face was used to authenticate and download the quantised GGUF model files. The model is then loaded from the GGUF file with llama-cpp-python , which runs it locally on the CPU. The implementation is not a Transformers-based inference pipeline. Table 5 gives the configuration.

Table 5: Local language model configuration

Setting	Value
Model	Llama 3 8B Instruct
Format	GGUF
Quantisation	Q4_K_M
Context window	4096 tokens
CPU threads	6
GPU layers	0 (CPU only)
Runtime	llama-cpp-python

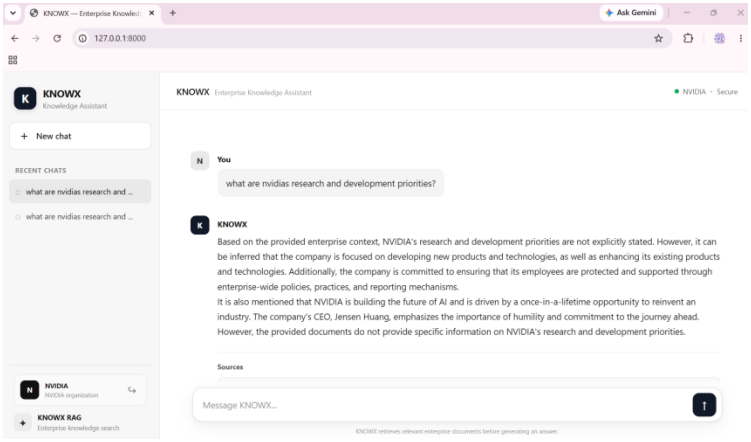
Table 6: Retrieval and generation parameters

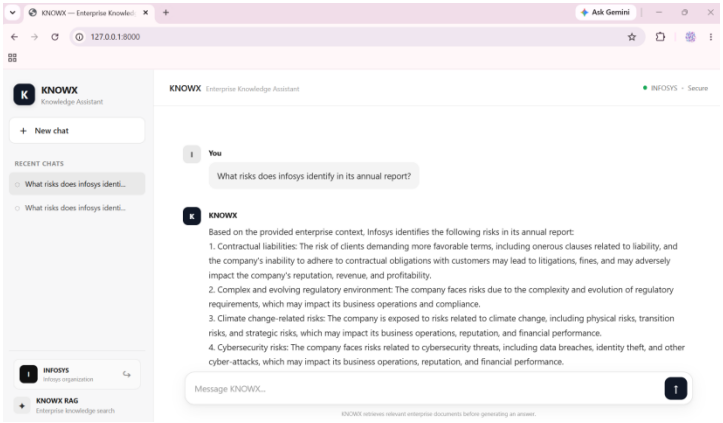
Parameter	Implementation
Chunking	Extracted text split into chunks of about 800 characters on average (5,000,829 characters into 6,256 chunks); each chunk keeps its document name and chunk identifier
Text cleaning before BM25	BM25 index built over the chunk text with rank-bm25; exact cleaning steps are not stated in this report
FAISS index	Stores the 384-dimensional all-MiniLM-L6-v2 embeddings in models/semantic_index.faiss; the index type and similarity setting are not stated in this report
Results taken from each retriever	Top-ranked chunks from both the FAISS and the BM25 retrieval are passed to fusion; the number is not stated in this report
RRF constant k	Constant in the score 1 / (k + rank); value not stated in this report
Cross-encoder model	cross-encoder/ms-marco-MiniLM-L-6-v2
Passages selected for the LLM context	Highest-scoring reranked passages, limited by the 4096-token context window; the number is not stated in this report
Prompt to the language model	Question together with the retrieved passages and their source information as context
Generation settings	Context window 4096, 6 CPU threads, 0 GPU layers (Table 5); other settings are not stated in this report

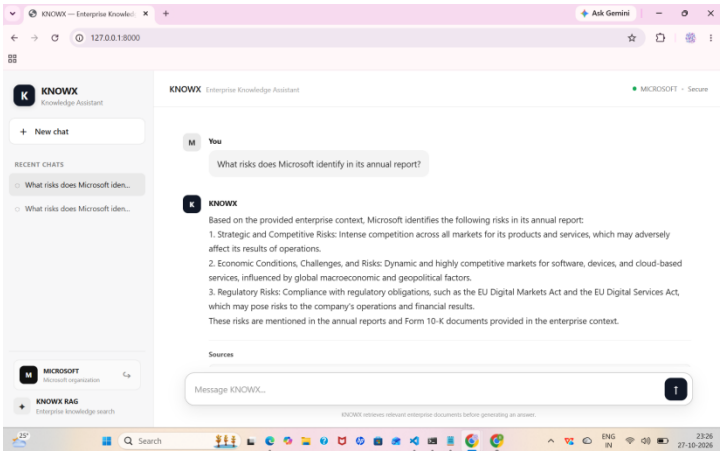
6.6 Demonstration and API

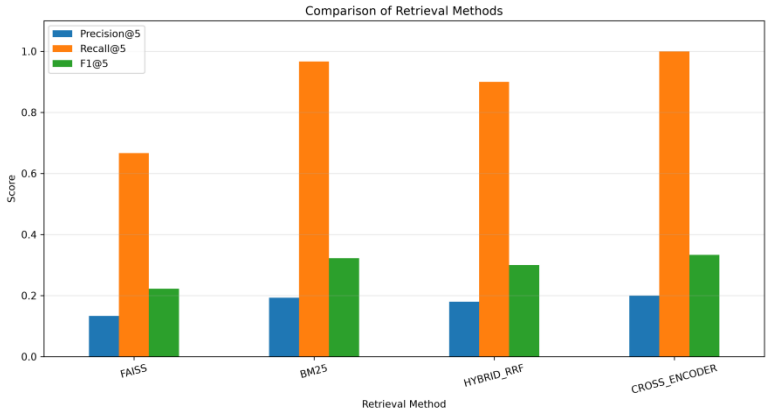
The demonstration interface is a FastAPI application in app/app.py, served with Uvicorn. A JavaScript front end in app/static/app.js sends the user's question to the backend and displays the generated answer together with its source entries. The backend passes each question to the retrieval and generation pipeline and returns the answer with the document name, chunk identifier and retrieval score of each source. The pipeline can also be run directly from the command line through src/pipeline.py, which is how the recorded test in Section 7 was run.

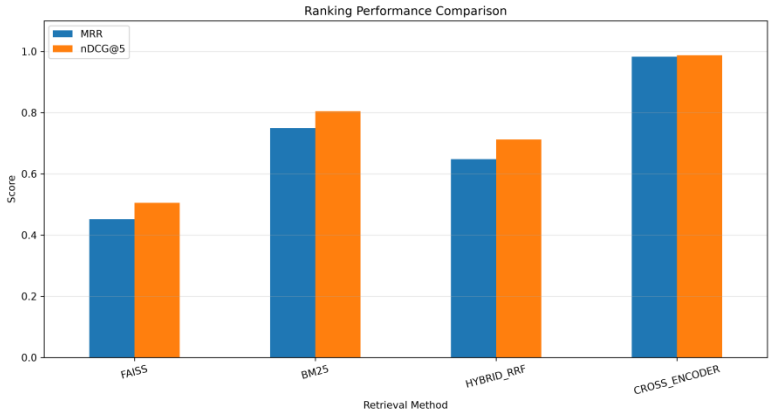
7. RESULTS AND DISCUSSION

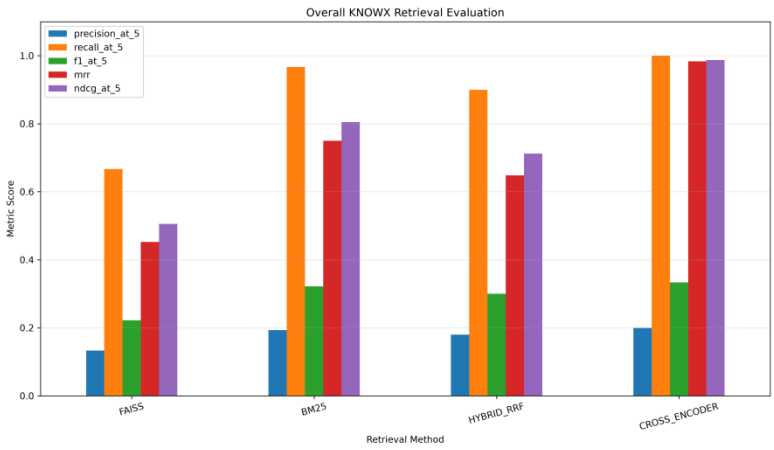


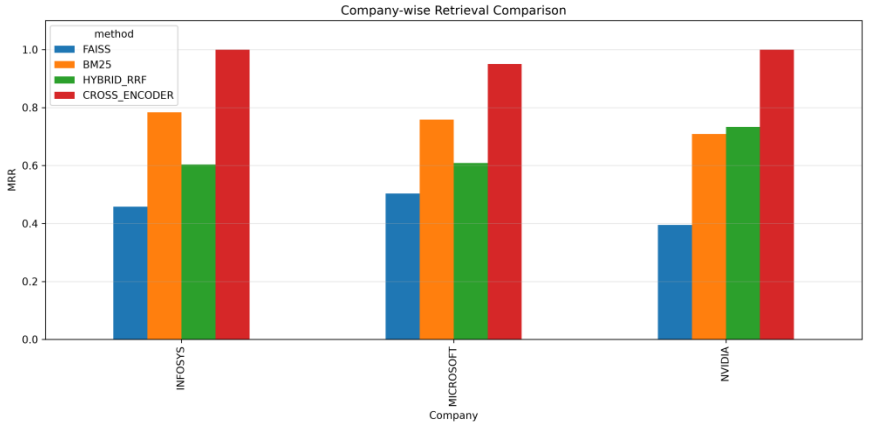


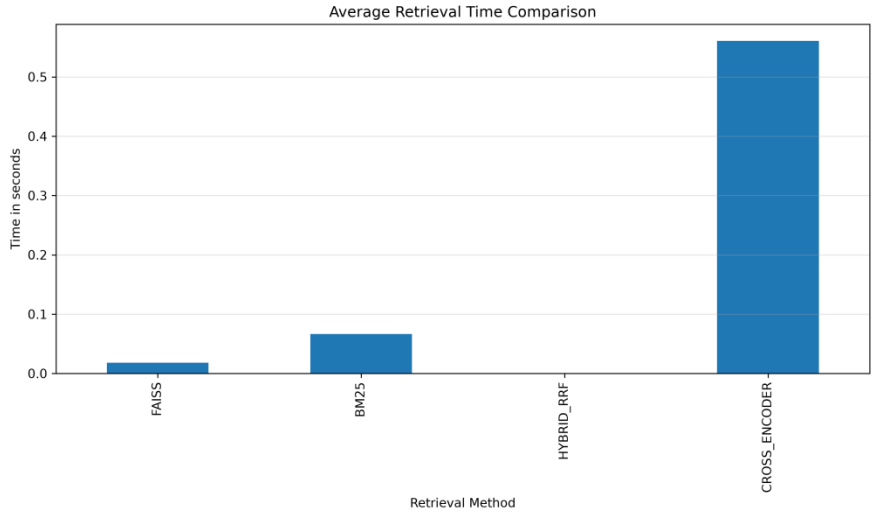



Comparison of Retrieval Methods
Precision@5
Recall@5
F1@5
Score
0.0
0.2
0.4
0.6
0.8
1.0
FAISS
BM25
HYBRID_RRF
CROSS_ENCODER
Retrieval Method


Ranking Performance Comparison
MRR
nDCG@5
Score
0.0
0.2
0.4
0.6
0.8
1.0
FAISS
BM25
HYBRID_RRF
CROSS_ENCODER
Retrieval Method


Overall KNOWX Retrieval Evaluation
precision_at_5
recall_at_5
f1_at_5
mrr
ndcg_at_5
Metric Score
0.0
0.2
0.4
0.6
0.8
1.0
FAISS
BM25
HYBRID_RRF
CROSS_ENCODER
Retrieval Method





Dataset summary

Table 7 summarises the collection and the indexes that were built. There is no class distribution to show because the documents carry no labels.

Table 7: Dataset and index summary

Item	Value
Document sources	Microsoft, NVIDIA and Infosys
Documents processed	9
Extracted text	Approximately 5,000,829 characters
Text chunks generated	6,256 (about 695 per document and about 800 characters per chunk on average, calculated)
Chunks from Microsoft	893
Chunks from NVIDIA	2,453
Chunks from Infosys	2,910

Item	Value
Embedding model and dimension	all-MiniLM-L6-v2, 384 dimensions
Semantic index	FAISS index built successfully
Keyword index	BM25 index built successfully

Recorded test of the integrated pipeline

The only recorded test of the integrated pipeline was run from the command line. Table 8 shows what it returned.

Table 8: Recorded test result

Item	Result
Test question	"What does Microsoft report about its business and financial performance?"
Answer	The pipeline generated an answer that discusses Microsoft's business and financial reporting
Sources returned	Five source entries, including MSFT_FY24Q4_10K.docx and Microsoft annual reports.docx
Information in each source entry	Document name, chunk identifier and retrieval score

This test confirms that retrieval, fusion, reranking, context building and local generation ran together without error. It does not show whether the answer is correct, because no reference answer was prepared.

Results that have not been measured and results that have been measured

A formal quantitative evaluation has not yet been performed. Table 9 lists the results that a text analytics report would normally contain and states why each is missing. No values were estimated or invented.Table 10 reports preliminary known-source retrieval benchmark results. The questions were automatically generated from selected document chunks, with one known relevant source chunk per question. These results are not independently validated accuracy measurements or a complete human-labelled relevance evaluation. Answer generation quality using Llama 3 has not yet been quantitatively evaluated.

Table 9: Standard results that are not available for KNOWX

Result	Status	Reason
Accuracy, precision, recall, F1-score	Not measured	No labelled test dataset exists. Measuring retrieval or answer quality would need questions with known relevant passages or reference answers
Confusion matrix and ROC curve	Not applicable	KNOWX returns ranked passages and generated text, not class predictions
Class distribution	Not applicable	The documents have no class labels
Train/validation/test split	Not applicable	No model was trained; all models are pre-trained
Comparison with baselines or other models	Not done	No baseline such as BM25-only, embeddings-only or no-reranker has been run
Response latency	Not measured	No timing benchmark was carried out
Statistical significance	Not tested	There are no repeated measurements to test

Table 10: Standard results that are available for KNOWX

Evaluation Metric	FAISS	BM25	Hybrid RRF	Cross-Encoder
Precision@5	0.1333	0.1933	0.1800	**0.2000**
Recall@5	0.6667	0.9667	0.9000	**1.0000**
F1-score@5	0.2222	0.3222	0.3000	**0.3333**
MRR	0.4522	0.7500	0.6483	**0.9833**
nDCG@5	0.5053	0.8049	0.7122	**0.9877**

The preliminary automated retrieval evaluation was conducted using 30 test questions across Microsoft, NVIDIA and Infosys. The Cross-Encoder achieved the highest scores across all five reported metrics, followed by BM25 and Hybrid RRF. These results represent a preliminary known-source retrieval benchmark and should not be interpreted as independently validated answer accuracy. Human-labelled relevance evaluation and quantitative answer-generation evaluation remain future work.

Discussion

The recorded test shows that the whole pipeline works from document retrieval to a generated answer on a laptop with no dedicated GPU. The source documents named in the record are both Microsoft documents, which fits the subject of the question. This is one test with one question and

no relevance judgement, so it gives no evidence about how often the retrieval finds the right passages or how accurate the answers are.

Several limitations follow from the current state of the work. The collection contains only nine documents from three organisations, so the system reflects what those documents cover. The documents were written by the companies themselves, so their wording and emphasis (a possible source of bias) carry over into the answers, and no other viewpoints are included. The answers come from a quantised (Q4_K_M) model run on a CPU, and the effect of this on answer quality and speed has not been measured. The 4096-token context window limits how many passages can be passed to the model. The reranker was trained on MS MARCO web-search passages , so its fit to long enterprise reports has not been tested. As with any RAG system, the model can still misread or overstate the retrieved text, which is why the sources are displayed so that users can check them. No error analysis was possible without labelled examples. On privacy, generation runs locally, so the question and the retrieved text are processed on the user's machine. Whether the documents may be shared or stored is a separate question that depends on their licence terms, which should be checked before the system is distributed.

8. CONCLUSION AND FUTURE SCOPE

This project built KNOWX, a question-answering system for enterprise documents. Nine documents from Microsoft, NVIDIA and Infosys were turned into 6,256 chunks and indexed twice, with 384-dimensional embeddings in FAISS and with BM25. A question is answered by hybrid retrieval, Reciprocal Rank Fusion, cross-encoder reranking, context building and generation with a locally run Llama 3 8B Instruct model. No models were trained, so no models were compared. The only recorded finding is that the integrated pipeline ran successfully from the command line and returned an answer with five sources for the test question.

Table 11: Status of the objectives

Objective	Status	Evidence
Collect and inspect the document collection	Achieved	9 documents, 6,256 chunks; no class distribution because there are no labels
Extract text and create chunks	Achieved	Ingestion completed
Create embeddings and a BM25 index	Achieved	FAISS and BM25 indexes built

Objective	Status	Evidence
Implement hybrid retrieval, RRF and reranking	Achieved	Implemented and integrated
Generate answers with local Llama 3 8B Instruct	Achieved	Model configured and run on CPU
Test the integrated pipeline	Partly achieved	One recorded test; no formal quantitative evaluation
FastAPI application and front end	Implemented	No interface test result is recorded
Discuss limitations, bias and privacy	Achieved	Section 7

The most important limitation is the lack of a quantitative evaluation. Until one is carried out, the quality of the retrieval and of the generated answers is unknown.

Future scope

- Build a labelled set of questions with known relevant passages and reference answers, and report retrieval metrics (for example Recall@k and MRR) and answer quality, including whether each answer is supported by its sources.
- Compare the full pipeline with simpler variants, namely BM25 only, embeddings only, fusion without reranking, and other local language models or quantisation levels.
- Measure response time and memory use on the CPU-only laptop and test the effect of the thread count and context size.
- Extend the collection to more documents and add a way to update the indexes when documents change, while watching for outdated answers.
- Improve explainability by highlighting the sentences that support each answer, add access control for confidential documents, and prepare the application for deployment.

9. REFERENCES

[1] Project team, “Enterprise document collection from Microsoft, NVIDIA and Infosys,” unpublished project dataset, 9 documents, collected and processed for the KNOWX project, 2026.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in Advances in Neural Information Processing Systems, vol. 33, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[3] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009. doi: 10.1561/1500000019.

[4] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), Hong Kong, China, 2019, pp. 3982–3992. doi: 10.18653/v1/D19-1410.

ACL Anthology

[5] G. V. Cormack, C. L. A. Clarke and S. Büttcher, “Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods,” in Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval, Boston, MA, USA, 2009, pp. 758–759. doi: 10.1145/1571941.1572114.

doi.org

[6] J. Johnson, M. Douze and H. Jégou, “Billion-scale similarity search with GPUs,” IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535–547, 2021. doi: 10.1109/TBDATA.2019.2921572.

IEEE Xplore

[7] R. Nogueira and K. Cho, “Passage Re-ranking with BERT,” arXiv:1901.04085, 2019. [Online]. Available: https://arxiv.org/abs/1901.04085.

arxiv.org

[8] Sentence Transformers, “Retrieve & Re-Rank,” Sentence Transformers Documentation. [Online]. Available: https://sbert.net/examples/applications/retrieve_rerank/README.html. Accessed: Oct. 30, 2026.

[9] Sentence Transformers, “all-MiniLM-L6-v2,” Hugging Face Model Card. [Online]. Available: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2. Accessed: Oct. 30, 2026.

[10] Facebook Research, “Faiss: A library for efficient similarity search and clustering of dense vectors,” GitHub Repository. [Online]. Available: https://github.com/facebookresearch/faiss. Accessed: Oct. 30, 2026.

GitHub+1

[11] Meta AI, “Introducing Meta Llama 3: The most capable openly available LLM to date,” Apr. 18, 2024. [Online]. Available: https://ai.meta.com/blog/meta-llama-3/.

ai.meta.com

[12] llama-cpp-python, “API Reference,” llama-cpp-python Documentation. [Online]. Available: https://llama-cpp-python.readthedocs.io/en/latest/api-reference/. Accessed: Oct. 30, 2026.

llama-cpp-python

[13] cross-encoder, “ms-marco-MiniLM-L-6-v2,” Hugging Face Model Card. [Online]. Available: https://huggingface.co/cross-encoder/ms-marco-MiniLM-L-6-v2. Accessed: Oct. 30, 2026.